

P2769R1

get_element customization point object

Published Proposal, 2023-05-18

This version:

<https://wg21.link/P2769R1>

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Alexey Kukanov](#) (Intel)

Audience:

SG9, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper introduces a CPO to read from tuple like objects, which can be used as projection for `std::ranges` algorithms.

Table of Contents

1 Motivation

- 1.1 Motivating use case
- 1.2 Projections-based alternative
- 1.3 The desired approach
- 1.4 Why not `std::ranges::views::elements`
- 1.5 Usefulness with `zip_view`
- 1.6 Radix sort use case

2 Proposed API

- 2.1 Possible implementation

3 Design considerations

- 3.1 What could be done to use `std::ranges::get` name

4 *tuple-like* concept

- 4.1 *tuple-like* concept generalization with `get_element`
- 4.2 *tuple-like* concept generalization with structured binding

5 Connections with other papers

- 5.1 Connection with [P2547R1]
- 5.2 Connection with [P2141R1]

6 Formal wording

- 6.1 Modify Header `<tuple>` synopsis [`tuple.syn`]
- 6.2 Add the following sections into [`tuple`]
- 6.3 Add the following wording into [`tuple.cust.get_elem`]
- 6.4 Add feature test macro to the end of [`version.syn`]

7 Revision history

7.1 R0 => R1

8 Polls

8.1 SG9 polls, Issaquah 2023

9 Acknowledgements

References

Informative References

§ 1. Motivation

§ 1.1. Motivating use case

Having `std::pair`, `std::tuple` and other *Tuple-Like* objects as the value types for the algorithms creates a plenty of opportunities. With special views, such as `std::ranges::elements_view`, we can specify which tuple elements to access when iterating over collections of such objects. However, we cannot easily use a predicate to make a decision based on only some of tuple elements, for example keys or values.

Let's consider the following example:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [](auto x, auto y)
{
    // key-based sorting
    return std::get<0>(x) < std::get<0>(y);
});
```

As we can see, users should spell some extra syntax out to achieve the necessary goal, comparing to what is described in [§1.3 The desired approach](#). The example above can be considered simplified; in real practice users might also need to think of e.g. adding references to lambda parameters to avoid copying.

The code above can be rewritten with *structured binding*:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [](auto x, auto y)
{
    // key-based sorting
    auto [key1, value1] = x;
    auto [key2, value2] = y;
    return key1 < key2;
});
```

Though one could say that it makes code simpler or at least more readable, on the other hand, its syntax forces the programmer to give names to otherwise unneeded variables, which is often considered a bad practice.

With [\[P2169R3\]](#) the situation with unused variables for structured binding becomes better but still might require the user to write a quite amount of underscores depending on the use case:

```
std::vector<std::tuple<int, int, int, int>> v{{3,1,1,1},{2,4,4,4},{1,7,7,7}};
std::ranges::sort(v, [](auto x, auto y)
{
    // key-based sorting
    auto [key1, _, _, _] = x;
    auto [key2, _, _, _] = y;
    return key1 < key2;
});
```

§ 1.2. Projections-based alternative

Projections provide another option to achieve the same behavior:

```
std::ranges::sort(v, std::less{}, [](auto x)
{
    // key-based sorting
    return std::get<0>(x);
});
```

A variant that properly handles references would use a generic lambda:

```
[](auto&& x) -> auto&&
{
    // key-based sorting
    return std::get<0>(std::forward<decltype(x)>(x));
}
```

While this code achieves the desired result, it requires more syntactic boilerplate (lambda, forwarding etc.) than the useful code.

§ 1.3. The desired approach

The nicest way to get what we want would be:

```
// The code that does not work because std::get is not fully instantiated
std::ranges::sort(v, std::less{}, std::get<0>);
```

But it doesn't work because `std::get` is a function template, and one cannot pass function templates as arguments without instantiating them.

§ 1.4. Why not `std::ranges::views::elements`

The necessary result cannot be achieved with `std::ranges::views::elements`, which would apply the filter for all operations on the input data, including element swap (for sort algorithm), while we need it to be only be applied for the comparator.

std::ranges::views::elements	Desired
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; // views::keys is an alias to views::elements std::ranges::sort(v std::ranges::views::keys, [](auto x, auto y) { return x < y; }); for (auto& x : v) { auto [key, val] = x; std::cout << "Key = " << key << ", Value = " << val << std::endl; }</pre> // Output (only keys are sorted): // Key = 1, Value = 1 // Key = 2, Value = 4 // Key = 3, Value = 7	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, [](auto x, auto y) { return std::get<0>(x) < std::get<0>(y); }); for (auto& x : v) { auto [key, val] = x; std::cout << "Key = " << key << ", Value = " << val << std::endl; }</pre> // Output (values are sorted by key): // Key = 1, Value = 7 // Key = 2, Value = 4 // Key = 3, Value = 1

§ 1.5. Usefulness with `zip_view`

With `std::ranges::zip_view` appearance in the standard the easy use of projection for *Tuple-Like* objects might become even more important because its dereferenceable type is exactly *Tuple-Like*.

§ 1.6. Radix sort use case

Counting-based sorts, and Radix Sort in particular, provide another motivating use case. Today it is not possible to have a C++ standard conformant implementation that uses Radix Sort algorithm underneath because the complexity of `std::sort` is defined as the number of comparator calls, while counting-based sorts do not use a comparator at all.

However, the industry needs Radix Sort for performance reasons. Implementations of C++ standard parallel algorithms, such as oneAPI Data Parallel C++ Library (oneDPL) and CUDA Thrust, use Radix Sort conditionally under the hood of `std::sort`, checking data types of the input and the comparator. In this case, a special comparator is of no help to sort values by keys, and projections seem the only viable option.

That makes the proposed API applicable wider than just with the C++ standard library use cases.

§ 2. Proposed API

We propose the following API:

```
inline namespace /* unspecified */
{
    template <size_t I>
        inline constexpr /* unspecified */ get_element = /* unspecified */;
}
inline constexpr auto get_key = get_element<0>;
inline constexpr auto get_value = get_element<1>;
```

With that API the motivating use case code with the desired behavior would be:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::get_element<0>);
```

or even

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::get_key);
```

Let's look at comparison tables (a.k.a. Tony Tables):

Comparison of proposed API with comparator-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, [](auto x, auto y) { return std::get<0>(x) < std::get<0>(y); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{ std::ranges::sort(v, std::less{}, st</pre>

Comparison of proposed API with projections-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, [](auto x) { return std::get<0>(x); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2, std::ranges::sort(v, std::less{}, std::ranges</pre>

§ 2.1. Possible implementation

```
namespace std
{
    namespace __detail
    {
        template <std::size_t _Ip>
        struct __get_element_fn
        {
            template <typename _TupleLike>
            auto operator()(_TupleLike&& __tuple_like) const ->
                decltype(get<_Ip>(std::forward<_TupleLike>(__tuple_like)))
            {
                return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
            }
        };
    } // namespace __detail

    inline namespace __get_element_namespace
    {
        template <std::size_t _Ip>
        inline constexpr __detail::__get_element_fn<_Ip> get_element;
    } // inline namespace __get_element_namespace

    inline constexpr auto get_key = get_element<0>;
    inline constexpr auto get_value = get_element<1>;
} // namespace std
```

§ 3. Design considerations

Alternative name for the proposed API could be `std::ranges::get`. Unfortunately, this name is already taken for `std::ranges::subrange` overload.

Potentially `std::ranges::get` could be repurposed for the proposed CPO with minimal API break for tricky scenarios only while still working as expected in existing reasonable use cases, as explained below. But we (likely) could not avoid an ABI break.

§ 3.1. What could be done to use `std::ranges::get` name

In all major standard library implementations (GCC, LLVM, Microsoft) the `get` overload for `std::ranges::subrange` is defined in `std::ranges`. Adding another definition of `get` to the same namespace would obviously create name ambiguity.

However, library implementors could move the current `std::ranges::get` function to an implementation specific namespace (e.g., `__detail`) and inherit (possibly privately) `std::ranges::subrange` from an empty tag class (e.g., `adl_hook`) defined in the same namespace. That way `std::ranges::__detail::get` can still be found by ADL for `std::ranges::subrange`, and at the same time, the `std::ranges::get` name becomes free to be redefined as a CPO that could successfully find a necessary overload for `get`, including the case when the argument is a `std::ranges::subrange`. Moreover, the proposed `std::ranges::get` CPO type could have a parameter pack in the interface to cover the use case when the current `std::ranges::get` function is used with explicit template arguments.

Please see the example that explains the idea and shows how it might look like. A full implementation with examples is available [here](#).

```
namespace std
{
    namespace ranges
    {
        // Necessary to make namespace __detail being considered by ADL
        // for get<0>(std::ranges::subrange<something>{}) without moving
        // the subrange itself to another namespace
        namespace __detail
        {
            struct adl_hook {};
        }

        // thanks to the empty-base optimization, inheriting adl_hook does not break ABI
        template <class T> class subrange : __detail::adl_hook {
            public: T whatever;
        };

        namespace __detail
        {
            template <std::size_t, class T>
            auto get(subrange<T> x) {
                return x.whatever;
            }
        } // namespace __detail
    } // namespace ranges

    using std::ranges::__detail::get;
} // namespace std

namespace std
{
    namespace ranges
    {
        namespace __detail
        {
            // Introduce Args... to cover the case of calling get with explicit template arguments
            template <std::size_t _Ip, typename... Args>
            struct __get_fn
            {
                // No more than std::tuple_size_v template arguments should be allowed
                template <typename _TupleLike> requires (sizeof...(Args) <= std::tuple_size_v<std::remove_cvref_t<_TupleLike>>() && __are_tuple_elements_convertible_to_args<std::remove_cvref_t<_TupleLike>>(), Args...>)
                decltype(auto) operator()(_TupleLike&& __tuple_like) const
                {
                    return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
                }
            };
        } // namespace __detail

        inline namespace __get_fn_namespace
        {
            template <std::size_t _Ip, typename... Args>
            inline constexpr __detail::__get_fn<_Ip, Args...> get;
        } // inline namespace __get_fn_namespace
    } // namespace ranges
} // namespace std
```

With such an implementation, all important cases from our perspective continue working:

- `std::ranges::get<0>(sub_r)`

- `std::get<0>(sub_r)`
- `get<0>(sub_r)`
- `std::ranges::get<0, some_arg>(sub_r)` (can also be used without namespace)

where `sub_r` is `std::ranges::subrange` object.

The API breaking change appears when `get` has all explicit template arguments for `subrange`, i.e., `std::ranges::get<Iarg, Sarg, Karg>(std::ranges::subrange<Iarg, Sarg, Karg>{})`. The problem is with the last `Karg` argument, which is unrelated to `tuple_size_v` and `tuple_element_t` of the subrange. Even if we say that the proposed backward compatible CPO with `Args...` does not constraint `sizeof...(Args)` and ignores the tail outside `tuple_size_v<subrange>`, it doesn't help for the mentioned use case because `K` of `std::ranges::subrange` is a non-type template parameter. Anyway, this scenario doesn't look common because explicit template parameters are used relatively rarely and furthermore, `K` has the default argument that is substituted based on a sentinel.

Definitely such a change would break the ABI for `std::ranges::get` because the fully qualified name of this function would change from what is in C++ standard library implementations today. But we think that that ABI break would have low impact because `std::ranges::get` is likely inlined and so doesn't create a visible symbol in translation units. We could imagine other tricky scenario where the API might be broken when `std::ranges::get` is used for something else but call. For example: `&std::ranges::get<0>`, `decltype(std::ranges::get<0>)`, etc. However, these scenarios don't look common.

Since the `std::ranges::subrange` API is relatively new, perhaps only a small amount of users would be affected but it can not be predicted accurately.

We would like to hear the opinion of the audience whether going with the `std::ranges::get` name is worth introducing potential inconvenience for some C++ users.

§ 4. tuple-like concept

With the proposed `std::get_element` CPO, the *tuple-like* concept can be generalized to cover wider range of types rather than just the listed standard types. Although we think it deserves a separate paper, yet we would like to provide some ruminations how it might look like.

§ 4.1. tuple-like concept generalization with `get_element`

The existing *has-tuple-element* exposition-only concept can be taken and modified with `get_element` in the following way:

```
template< class T, std::size_t N >
concept /*has-tuple-element*/ = // exposition only
    requires(T t) {
        typename std::tuple_size<T>::type;
        requires N < std::tuple_size_v<T>;
        typename std::tuple_element_t<N, T>;
        // std::get_element substitutes for std::get, the rest is the same
        { std::get_element<N>(t) } -> std::convertible_to<const std::tuple_element_t<N, T>>
    };

```

With that modification the old code continues to work. We relax the concept. Since it's exposition only and, to the best of our knowledge, doesn't affect overload resolution or class specialization then it should not cause unexpected overloads being called.

Then the *tuple-like* concept can reuse *has-tuple-element* and do something like:

```
// necessary to check if std::tuple_size_v is well-formed before using it
template <typename T>
concept /*has-tuple-size*/ = // exposition only
    requires {

```

```

    typename std::tuple_size<T>::type;
};

template <typename T>
concept /* tuple-like */ = !std::is_reference_v<T> &&
    /*has-tuple-size*/<T> &&
    [<std::size_t... I>(std::index_sequence<I...>) {
        return (... && /*has-tuple-element*/<T, I>);
    } (std::make_index_sequence<std::tuple_size_v<T>>{}));

```

§ 4.2. *tuple-like* concept generalization with structured binding

Another way to generalize the *tuple-like* concept is via structured binding. Basically it's proposed in [\[P2141R1\]](#). Please see [§ 5.2 Connection with \[P2141R1\]](#) for more information.

§ 5. Connections with other papers

§ 5.1. Connection with [\[P2547R1\]](#)

[\[P2547R1\]](#) uses `std::get` as the example and a good candidate to be a customizable function. Authors plan to ship the customizable functions proposal first and deal with customizing standard library functions later. That means we should not expect that examples in this paper automatically would be transformed to customizable functions when it will land.

Moreover, at this time the authors of [\[P2547R1\]](#) don't see how to introduce customizable functions with the same names (e.g. `std::get`) without the ABI break, so they will likely need to choose different names.

§ 5.2. Connection with [\[P2141R1\]](#)

[\[P2141R1\]](#)'s main goal is allow aggregates being interpreted as *Tuple-Like*. At the same time, it touches the *tuple-like* concept making it as generic as for the types structured binding can work with. It also adds a yet another `std::get` overload that works with any *Tuple-Like* object except those that are already in the `std::` namespace.

With [\[P2141R1\]](#) being adopted `std::get` does the right thing and works with *Tuple-Like* object, so we may use just `std::get<_Ip>(std::forward<_TupleLike>(__tuple_like))` within the implementation of `std::get_element` instead of the unqualified `get` call.

Independently of [\[P2141R1\]](#) `std::get_element` brings its own value by covering the described motivation use-cases. Furthermore, in the standard there are already precedences of having two similar things with slightly different semantics, for example, `std::less` and `std::ranges::less`, where the latter is not even a CPO.

§ 6. Formal wording

Below, substitute the  character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 6.1. Modify Header `<tuple>` synopsis [\[tuple.syn\]](#)

```
[...]
// [tuple.helper], tuple helper classes
template <class T>
    constexpr size_t tuple_size_v = tuple_size<T>::value;

inline namespace /* unspecified */ {
    template <size_t I>
        inline constexpr /* unspecified */ get_element = /* unspecified */;
}
inline constexpr auto get_key = get_element<0>;
inline constexpr auto get_value = get_element<1>;
```

§ 6.2. Add the following sections into [\[tuple\]](#)

```
[...]
◆ Element access \[tuple.elem\]
◆ Customization Point Objects \[tuple.cust\]
◆ get_element \[tuple.cust.get\_elem\]
```

§ 6.3. Add the following wording into [\[tuple.cust.get_elem\]](#)

1. The name `get_element` denotes a customization point object ([\[customization.point.object\]](#)). The expression `get_element<I>(E)` where `I` is `std::size_t` for a subexpression `E` is expression-equivalent to:
 1. `get<I>(E)`, if `E` has class or enumeration type and `get<I>(E)` is a well-formed expression when treated as an unevaluated operand, where the meaning of `get` is established as-if by performing argument-dependent lookup only ([\[basic.lookup.argdepl\]](#)).
 2. Otherwise, `get_element<I>(E)` is ill-formed.

§ 6.4. Add feature test macro to the end of [\[version.syn\]](#)

```
[...]
#define __cpp_lib_element_access_customization_point 200000L
// also in <tuple>, <utility>, <array>, <ranges>

[...]
```

§ 7. Revision history

§ 7.1. R0 => R1

- Address the "structured binding unused variables" questions with [\[P2169R3\]](#)
- Add ruminations about possible relaxation of the *tuple-like* concept
- Add connections to [\[P2547R1\]](#) and [\[P2141R1\]](#)
- Apply an approach to minimize ABI and API breaking for the `std::ranges::get` name
- Add wording and a feature test macro for the `std::get_element` API

§ 8. Polls

§ 8.1. SG9 polls, Issaquah 2023

POLL: The solution proposed in the paper "P2769: [get_element](#) customization point object" should be renamed to `std::ranges::get`.

SF	F	N	A	SA
1	2	1	2	1

POLL: The solution proposed in the paper "P2769: [get_element](#) customization point object" should be moved out of the `ranges` namespace (`std::get_element`).

SF	F	N	A	SA
2	4	0	1	0

§ 9. Acknowledgements

- Thanks to Casey Carter for providing [adl_hook](#) trick for [§ 3 Design considerations](#)
- Thanks to Corentin Jabot for providing the input for [§ 5.1 Connection with \[P2547R1\]](#) and for the initial *tuple-like* concept relaxing idea
- Thanks to Benjamin Brock for paper review and design discussions.

§ References

§ Informative References

[P2141R1]

Antony Polukhin. [Aggregates are named tuples](#). 3 May 2023. URL: <https://wg21.link/P2141R1>

[P2169R3]

Corentin Jabot, Michael Park. [A Nice Placeholder With No Name](#). 15 December 2022. URL: <https://wg21.link/p2169r3>

[P2547R1]

Lewis Baker, Corentin Jabot, Gašper Ažman. [Language support for customisable functions](#). 16 July 2022. URL: <https://wg21.link/p2547r1>